

ReBAC V3.3 — Compiler Enhancements (Query Planner, Cache, Profiler)

Branch: `ReBAC-V3.3` | **Builds on:** *V3.2 Language Evolution*

The Problem V3.2 Left Unsolved

The ADSL compiler now generates a correct, optimized RuleGroup IR. But it evaluates rules **in source order** — the order they appear in the `.auth` file:

```
permission view =  
  contains -> view      ← 30 recursive DB lookups  
  OR doctor_of          ← 1 direct DB lookup
```

The engine does the expensive traversal *first* and the cheap lookup *second* — purely because of source order. This is the authorization equivalent of PostgreSQL choosing a full table scan over an index scan because of column order in a `WHERE` clause.

V3.3 adds intelligence to the execution layer: Query Planner, Incremental Compilation Cache, and EXPLAIN ANALYZE Profiler.

V3.3 Architecture

1. Cost Estimator

```
class CostEstimator {  
  estimate(rule: Rule): number {  
    if (!rule.permission) return 1;           // Direct lookup - 0(1)  
    SQL  
    if (rule.relation === "contains") return 10 + depth * 5; //  
    Recursive  
    return 2;                                 // Computed userset
```

```
}  
}
```

Assigns static cost weights:

- **DirectLookup**: cost = 1 (single indexed SQL query)
- **ComputedUserSet**: cost = 2 (one permission resolution)
- **RecursiveTraversal**: cost = 10 + (5 × estimated depth)

2. Query Planner — Cheap-First Reordering

```
class QueryPlanner {  
  reorder(group: RuleGroup): ExecutionPlan {  
    // For OR groups: sort by ascending cost (short-circuit on  
    cheapest first)  
    // For AND groups: sort by ascending cost (fail fast on  
    cheapest first)  
    const sorted = group.rules.sort((a, b) =>  
      costEstimator.estimate(a) - costEstimator.estimate(b)  
    );  
    return { operator: group.operator, steps: sorted };  
  }  
}
```

The optimization:

```
BEFORE: contains.view (cost=30) OR doctor_of (cost=1)  
AFTER:  doctor_of (cost=1) OR contains.view (cost=30)
```

If `doctor_of` is true → short-circuit → `contains.view` never runs → 29 DB lookups saved.

This is exactly how PostgreSQL's cost-based optimizer works — it reorders join and scan operations by estimated cost.

3. Incremental Compilation Cache

```
class CompilationCache {  
  private store = new Map<string, RebacSchema>();  
  
  computeHash(source: string): string {
```

```

    return createHash("sha256").update(source).digest("hex");
  }

  get(hash: string): RebacSchema | undefined { ... }
  set(hash: string, schema: RebacSchema): void { ... }
}

```

SHA-256 content-addressed caching:

- On startup: hash each `.auth` file's content
- If hash exists in cache → skip compilation (return cached schema)
- If hash changed → recompile only that file

This is the same approach used by TypeScript's `--incremental` mode, Webpack's file hashing, and Bazel's build cache.

4. EXPLAIN ANALYZE Profiler

```

interface ExecutionProfile {
  totalTimeMs: number;
  dbLookups: number;
  maxRecursionDepth: number;
  memoHits: number;
  ruleBreakdown: RuleMetric[];
}

```

Unlike V2.3's static Explain Plan, the V3.3 profiler instruments **actual runtime execution**:

EXPLAIN ANALYZE patient.view for user=Raj, resource=Patient101

Total time:	12.23ms
DB lookups:	1 (out of 3 possible)
Recursion depth:	2
Memo hits:	0

Rule breakdown:

doctor_of	0.82ms → true (short-circuited)
assigned_nurse	[skipped – OR short-circuit]
contains.view	[skipped – OR short-circuit]

Critical Review

✓ What V3.3 Gets Right

1. **Cost-based query planning is the correct approach.** Every production database query optimizer (PostgreSQL, MySQL, BigQuery) uses cost estimation + rule reordering. Applying this to authorization is novel and correct.
2. **Content-addressed caching (SHA-256)** is the industry standard for build and compilation caching (Bazel, Buck, Nix, Docker layer caching).
3. **EXPLAIN ANALYZE** with actual runtime metrics is the right tool. SpiceDB exposes this via its `ExperimentalReachableResources` API with timing data.

⚠ What V3.3 Is Still Missing

Missing: Runtime cost feedback loop The `CostEstimator` uses static weights. A real optimizer tracks **actual measured latencies** and updates cost estimates dynamically — like PostgreSQL's `pg_statistic` table feeding the planner.

Missing: Parallel execution plan The planner reorders rules but executes them **serially**. Zanzibar evaluates multiple check paths in **parallel** (goroutines in Go) and returns on the first `true`. For OR groups with equal-cost branches, parallel execution would reduce P99 latency significantly.

Missing: Query plan caching The query plan is recomputed on every `checkPermission` call. Plans should be cached per `(resourceType, permission)` pair.

Missing: Adaptive timeout No deadline/timeout propagation through the execution plan. A single recursive traversal can block indefinitely.

✗ Incorrect: Static Cost Model

The cost model assigns `contains = 10 + depth * 5` but this is a guess. In production:

- A `contains` traversal on a shallow 2-level hierarchy: cost ≈ 2
- A `contains` traversal on a 10-level hierarchy: cost $\approx 50+$

The static model cannot distinguish these without runtime statistics. The correct approach is to seed the model with histogram statistics from past query executions — exactly like `ANALYZE TABLE` in PostgreSQL.

Summary

V3.3 adds the three components that separate a compiler from a **production-grade compiler**: cost-based query planning, incremental compilation caching, and a real-time EXPLAIN ANALYZE profiler. The designs are architecturally sound and mirror PostgreSQL's optimizer, Bazel's build cache, and SpiceDB's execution tracing. The main gaps are the static cost model (should be adaptive) and serial execution (should be parallel for OR branches).